

Privilege on the Exit Path: A Name-Agnostic Static Detector and a Tiered Benchmark for Malicious Solidity Contracts

20 September 2026

Status: Final candidate, unapproved
Date: 2026-09-20
Authors: Team EarthIsMine [check: author list and affiliation]
Artifacts: `detector/` (tool), `baybench/` (harness), `cases/` (corpus), `reports/` (frozen runs)
Measurements: `reports/{detector,noexit,baseline_slither,baseline_keyword}/report.json` at commit 799361b; derived statistics from its 859 per-case rows by `docs/research/_paper_stats.py`
Specifications: [baybench.md](#), [detector.md](#)
Taxonomy: [track1-malice-patterns.md](#)

Abstract

Token scams are not accidental bugs. The attacker writes the contract, deploys it, and then uses a privilege the source code always contained. Detecting that intent from source is a different problem from finding vulnerabilities, and the published detectors for it converge on one structural question: is there a privileged writer to a state variable that sits on the user’s transfer or exit path, or that touches balances directly? This paper reports an implementation of that predicate and the benchmark built to measure it. The detector runs name-agnostic predicates on Slither’s intermediate representation, carries 29 rule identifiers across seven pattern families, adjusts each finding by a two-class discriminator layer that separates bounded power from unbounded power, and resolves the finding set through a three-step decision ladder that abstains only when the analysis genuinely cannot see the answer. The benchmark, BAYBENCH, scores any black-box tool over 859 labelled Solidity files in four tiers, offline and under `docker run --network none`. On that corpus the detector reaches a tier-weighted score of 0.9714 against 0.6298 for native Slither high-impact findings and 0.4269 for a name-regex baseline, with a HIGH false-positive rate of 0.0 over the 42 benign-preferred files and an exact 5 of 5 on the organizers’ public sample set. We report what the measurement does not support: family recall is any-hit recall against a single paper tag, the Tier 1 fixtures were written by the authors of the rules, and several corpus labels were amended in the same change set as the decision policy.

Keywords: smart contract security, rug pull detection, honeypot detection, static analysis, Solidity, benchmark

1. Introduction

A rug pull is a contract that works exactly as written. The deployer retains a function that mints without a cap, blocks a holder’s transfer, redirects a fee, or delegates execution to an address it can change later. Nothing about that code is a memory-safety error, an arithmetic oversight, or a reentrancy window. It is an intentional capability, disclosed in the source, exercised when the liquidity is deep enough to be worth taking. Measurement studies put the scale in the tens of thousands of tokens and hundreds of millions of dollars, with roughly half of Uniswap V2 listings in one 2022 study classified as scams [14], and 60 percent of new tokens in a 2023 study living less than a day [15].

This paper describes a detector and a benchmark built for TRUST404 Track 1, a competition task whose brief states the problem class precisely: identify risk from logic flow and permission structure rather than keyword matching, and reason about abnormal transfer of privilege and concealed asset withdrawal paths [20]. Those two phrases are the literal names of two categories in the rug-pull literature, Ownership Fraud and Leaking Token. The task is scam-contract detection, not vulnerability detection, and the distinction changes what a correct answer looks like. A contract with an owner-only mint function has no bug. Whether it is malicious depends on whether the mint is bounded by code, whether the bound is enforced symmetrically, and whether the privilege reaches an asset the user owns.

1.1 Problem

Two properties of the task make it hard for the obvious approaches. First, names lie. The literature documents blacklists named `_isBot`, backdoors named `failsafe` or `safeWithdraw`, and one-letter switches [4][6][12]. A checklist of identifier regexes therefore fails in both directions: it flags every legitimate `pause` and misses every renamed gate. Second, the benign lookalike is common and ordinary. `OpenZeppelin Pausable`, `ERC20Capped`, and `AccessControl` produce exactly the shapes a naive privilege detector reports, and a tool that calls them malicious is unusable on real code. A published systematization of 35 rug-pull types found that 13 evaluated tools covered between 25.7 and 62.9 percent of them, that 9 types were detected by no tool at all, and that on compound scams the best tool dropped to 31.3 percent [10].

1.2 Contributions

This paper makes four contributions.

1. **A consolidated predicate and its name-agnostic implementation.** We reduce nine published detectors to one triple of privilege, writable state, and impact on the transfer path, and implement it on Slither’s intermediate representation with no identifier word lists. A renaming test enforces the property mechanically.
2. **A decision layer that separates bounded from unbounded privilege.** Findings are adjusted by two disjoint discriminator classes. Bounding discriminators, which show the power is limited by code or does not reach a user asset, demote a finding to evidence. Governance discriminators, which show only who holds the power, annotate the reason and never change the verdict. The distinction is what keeps a capped mint benign and an uncapped one malicious.
3. **BAYBENCH, a tiered offline benchmark.** 859 labelled Solidity files in four tiers, a label schema that records a preferred verdict and a set of accepted verdicts, per-tier and per-family metrics, a determinism check, and grading parity fixed at `docker run --rm --network none`. Any tool that reads a directory and writes one `results.json` is scored identically.

4. **A measured evaluation with its limits stated.** We report the detector against a second independent implementation and two baselines on the same 859 files, give a verdict confusion matrix per tier, quantify the policy change that lifted the honeypot family, and enumerate the construct-validity threats that the headline number hides.

1.3 Research questions

The evaluation in Section 6 answers five questions. RQ1: how does the structural detector compare with a native static-analyzer baseline and a name-matching baseline on the same corpus? RQ2: where does its accuracy come from, by tier and by pattern family? RQ3: what is its false-positive behaviour on contracts that are benign but look risky? RQ4: how much of the result is the decision policy rather than the rule set? RQ5: does it satisfy the operational constraints of offline, deterministic, non-interactive grading?

2. Background and Related Work

Three sub-literatures study deployer intent rather than accidental defects: honeypots from 2019, token backdoors and rug pulls from 2022, and trapdoor tokens from 2023. They differ in corpus and in analysis substrate, and they agree on the predicate.

2.1 Honeypots and token backdoors

HoneyBadger [1] produced the first honeypot taxonomy, eight techniques grouped into three levels according to whether the trap exploits the EVM, the Solidity compiler, or the way Etherscan displays a contract. It used symbolic execution over bytecode, flagged 690 honeypots, and reported 87 percent precision on manual validation. Its appendix lists 24 named honeypots with addresses, which makes it directly usable as a fixture source.

Pied-Piper [2] defined five ERC-token backdoors: arbitrary transfer, generate token after ICO, destroy token, disable transferring, and freeze account. It combined a Datalog formulation with directed fuzzing, confirmed 189 backdoors in 13,484 contracts, and produced four CVEs. It also released 200 backdoor-injected contracts as a test set.

TokenScope [3] took a different angle, detecting inconsistency between storage changes, interface behaviour, and emitted events, for instance a mint with no `Transfer` event or a `balanceOf` whose return value contradicts storage. It reports precision above 99.9 percent. The check that a view function must not branch on its caller descends from this work.

2.2 Static rug-pull detection

CRPWarner [4] is the closest published system to the present one. It collected 103 real rug pulls, reduced the contract-related cases to three types (hidden mint, limiting sell order, leaking token, with unlimited fee as a variant), and expressed each as a Datalog rule over bytecode. Its hidden-mint rule is a conjunction of a public function reachable only by the owner, a load-and-store on the balance mapping, and the absence of a balance check. On 69 open-source scam contracts it reports precision 91.8, recall 85.9, and F1 88.7. Tokeer [5] analysed 201 incidents totalling 425 million dollars and derived four risks with base rates: blacklist at 58.7 percent, balance modification at 42.3 percent, time limits, and dependence on an external contract. Its contribution to modelling is the insight that a gate may sit anywhere on the reachable path, including before the internal transfer function is entered and inside helper functions two or three calls

deep. It reports 98.0 percent recall and 98.9 percent precision on its corpus, and finds 27.2 percent more real risks than the tools it compares against.

TrapdoorAnalyser [6] manually inspected 1,859 trapdoor tokens, named five techniques, and, importantly for this work, built its semantic check on Slither’s AST through the Python API. Its formulation collects state variables, the transfer functions and everything they call, the transfer inputs, the end nodes of those functions, the backdoor functions in which the caller is compared with a stored address or list, and the external calls. An indicator is a state variable used in an end node of the transfer path, or in the amount computation, that is writable from a backdoor function or an external call. That is the same shape implemented in Section 4.

RPHunter [7] assembled the largest manual corpus with source, 645 rug pulls, and organised code risk into three categories and eight sub-types: sale restrict (203 cases), variable manipulation (144), and balance tamper (189). Later work tracks balance changes per function on bytecode across six backdoor types [8], and a taint-based honeypot detector keys on boolean storage indexed by sender or recipient that decides a revert branch [9]. The latter is instructive in its failure: it missed the Pied-Piper backdoors that never touch `transfer`, which is direct evidence that exit gating alone is an insufficient model.

2.3 Systematizations and obfuscation

The ISSA 2025 systematization [10] merged 31 papers and 27 industry sources into 35 rug-pull types and evaluated 13 tools against them. Beyond the coverage numbers already cited, it names four failure modes of static tools: blindness to dynamic context such as timestamp-gated behaviour, failure to resolve hidden variables such as dynamically assigned owner addresses, complex multi-condition gates, and governance-functionality decoupling, where the authorisation check and the balance operation live in different functions. Its case study on DOGE3.0 combines a proxy that hides ownership, a hidden `_sender` role, and a burn function that mints. An earlier systematization catalogued 34 root causes and listed types no tool covered [11].

ObfProbe [12] measured seven bytecode-level obfuscation features across 1.03 million contracts and found 3,128 heavily obfuscated ones. Its most relevant result for evaluation design is that a state-of-the-art Ponzi detector fell from 79 percent to 12 percent under obfuscation. A large-scale Slither study of 49.9 thousand NFT contracts [13] reports which constructs dominate flagged contracts: owner-only withdraw, unrestricted mint, `selfdestruct`, `delegatecall`, and `tx.origin` authorisation.

Industry checklists [18][19] expose the same categories as boolean fields, for instance `is_honeypot`, `hidden_owner`, `is_mintable`, `transfer_pausable`, and `owner_change_balance`. They are the approach a keyword baseline approximates, and the ISSA evaluation is the reason to expect them to be brittle.

2.4 Positioning

Three properties distinguish the present work from the systems above. It analyses source rather than bytecode, which means the operator that distinguishes a mint from a burn is visible rather than inferred from a compiler rewrite, a false-positive source CRPWarner documents explicitly. It refuses identifier matching as a mechanical property rather than a design intention, and tests that refusal. And it treats the verdict as a separate layer from the findings, which is what allows a capped mint and an uncapped mint to produce the same finding and different answers. The accuracy figures quoted above are each measured on their authors’ own corpora and are not comparable with the numbers in Section 6; they are context for the design, not a baseline.

3. Threat Model and Pattern Taxonomy

3.1 Scope and assumptions

The analysed artefact is Solidity source, supplied as a directory, analysed offline with no chain access, no network, and no hosted model. The adversary is the deployer. The adversary controls the source, may rename every identifier, may place a check in a modifier defined in another contract or in a helper several calls deep, and may condition the trap on block timestamp or on a specific recipient. The adversary cannot change the semantics of the ERC-20 interface that a wallet or an exchange calls, and cannot hide a state write from the compiler. Those two residual anchors are what the detector uses.

Out of scope: anything that requires transaction history, liquidity-pool state, or holder distribution, all of which the offline constraint removes; and accidental vulnerabilities, which are the subject of the mature tooling this work uses as a library rather than as a competitor.

3.2 Pattern families

The taxonomy merges HoneyBadger, Pied-Piper, CRPWarner, Tokeer, TrapdoorAnalyser, RPHunter, the ISSTA systematization, and the two industry checklists, and groups patterns by what the attacker gains rather than by which paper named them. Seven families result, labelled A to G, and every rule and every corpus label in this work is expressed in them.

Family	Name	Representative shapes	The attacker gains
A	Exit gating	Address allowlist or blocklist read in a revert on the transfer path; global suspension switch; amount limit with no floor; cooldown; sell-only branch; unbounded fee	The holder cannot sell
B	Balance tamper	Privileged mint with no cap; burn from another account; direct assignment to the balance mapping; credit exceeding debit inside transfer; a view function that branches on its caller	A larger share of supply
C	Fund extraction	Arbitrary <code>transferFrom</code> without allowance; a privileged branch in transfer that skips the debit; a privileged sweep of contract-held value; a mutable fee recipient	Custodied value leaves
D	Control-plane deception	A second address variable used in authorisation while ownership is visibly renounced; a renounce that does not clear every role; non-standard reassignment; <code>tx.origin</code> authorisation	A role nobody can see
E	Structural escape hatch	Transfer gate delegated to a settable external contract; <code>delegatecall</code> to a settable or caller-chosen target; reachable <code>selfdestruct</code> ; upgradeable proxy under a single key	Behaviour outside the file
F	Honeypot and drainer	The eight HoneyBadger techniques; an approval-pull contract that moves tokens from the caller with no credit back	The deposit or the approval
G	Ponzi	Payout to stored depositors funded only by <code>msg.value</code> , with no external value source	Later deposits

Base rates justify the weight the corpus and the rule set give to A and B. Tokeer’s incident study puts blacklist at 58.7 percent and balance modification at 42.3 percent; RPHunter’s 645 sources contain 203 sale-restrict and 189 balance-tamper cases [5][7]. Families C through G are individually rarer but are where the published tools disagree most.

3.3 Benign lookalikes

Half the difficulty is on the other side. Seven shapes recur in legitimate code and produce the same syntactic evidence as a family A or B pattern. A burn is a mint with the opposite operator, and the operator is visible only in source. A fee-exemption map and a blacklist are the same `mapping(address => bool)`, distinguished by whether the read feeds an amount computation or a revert. A comparison against a DEX pair address looks like an authorisation check but has no `msg.sender` in it. An `address => uint` status map is not a balance mapping unless `balanceOf` reads it. `OpenZeppelin Pausable`, `Ownable`, `ERC20Capped`, and `AccessControl` are ubiquitous and bounded. An anti-bot launch limit with a hard-coded floor and a time-limited window is not an amount trap. A rescue function for foreign tokens is not a sweep of user funds. Each of these becomes a discriminator in Section 4.5. Their presence in the corpus as Tier 3 is what stops the evaluation from rewarding an over-eager detector.

4. Detector Design

4.1 The unifying predicate

Every strong tool in Section 2 reduces to the same triple, and the detector implements it directly.

A finding requires a privileged writer, a state variable that writer can set, and an impact: the variable is read in a revert-bearing node on the transfer path, feeds the transferred or fee amount, is the balance or supply ledger itself, or is the destination of an external call on that path.

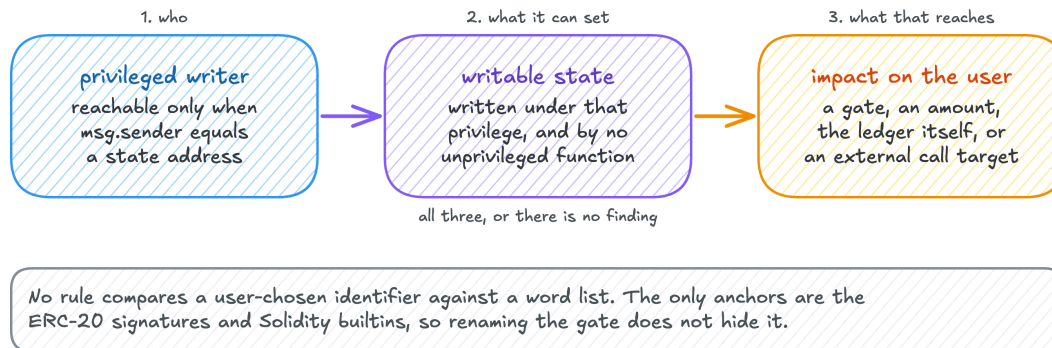


Figure 1: The three terms are computed independently. A finding needs all three; any two of them describe ordinary code. The three terms are independently computed, and each is available from Slither’s intermediate representation. Privilege comes from authorisation atoms in the reachable condition set. Writability comes from the set of state writes in privileged functions minus the set of variables any unprivileged function can write. Impact comes from the transfer-path closure, the balance binding, and the value-flow helpers. The detector is roughly 9,600 lines of Python across the analysis layer, the rule modules, and the engine.

4.2 Name-agnostic privilege

The judging criterion and the literature agree that identifier matching is the primary failure mode, so the implementation forbids it as a property rather than a convention. No rule or analysis module compares a user-chosen identifier against a word list. The only identifier-shaped anchors permitted are protocol

constants: the ERC-20 function signatures, the ERC-2612 `permit` signature, and Solidity builtins such as `msg.sender`, `tx.origin`, `block.timestamp`, `selfdestruct`, and `delegatecall`.

An authorisation atom is a comparison or lookup in a node's condition that is data-dependent on `msg.sender` or `tx.origin` and resolves against state: an equality or inequality against a state address variable, or a boolean loaded from a state mapping, possibly nested, indexed by a sender-dependent key. Two exclusions carry most of the lookalike burden. A check of the form `require(balances[msg.sender] >= amount)` is not an atom, because the value is not boolean and the comparison is not against an address. A comparison of the recipient against a stored pair address is not an atom, because no sender appears in it. Atoms are collected over the function body, its modifiers, and its internal callees, which is what catches the check placed in a modifier defined in another contract and the check wrapped in a boolean helper.

A privileged write is a state write in a privileged function, or a state write inside a branch guarded by an authorisation atom, excluding constructors. A variable is privileged-writable when at least one privileged write reaches it and no unprivileged function writes it, with `constant` and `immutable` variables excluded. One refinement proved necessary on real code: a gate also qualifies when any privileged writer can store the blocking polarity, even if an unprivileged path can also write the variable, because a contract that auto-flags addresses in `_transfer` while the owner can also flag anyone is still owner-controlled.

A role is hidden when it is an authorisation variable that is neither public nor reachable through the return value of any public view function. That definition, not a name, is what identifies the `_sender` and `isTokenReceiver` shapes the ISSTA case study reports.

4.3 Transfer-path closure and balance binding

The transfer path is the call-graph closure from the ERC-20 entry points and their modifiers, following internal calls, which is Tokeer's model. End nodes are the `require`, `assert`, `revert`, and conditional-branch nodes in that closure. A gate read is a read of a state variable in an end node, classified by shape: a boolean from an address-keyed mapping, a plain boolean, a numeric compared against the amount, a numeric compared against a time source, or an address compared against the sender or recipient. Those five shapes are the triggers of the family A rules.

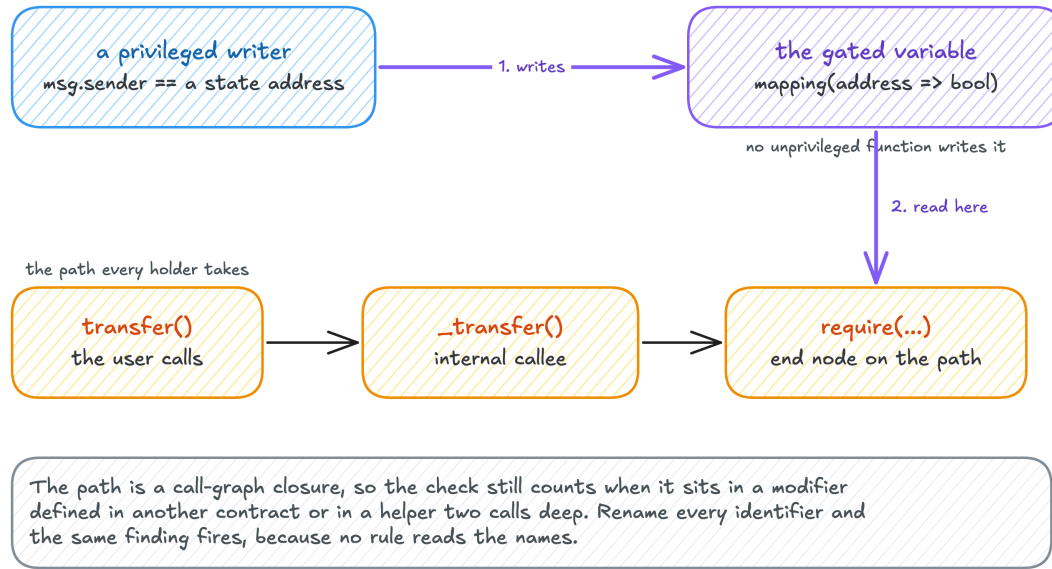


Figure 2: The same three terms on a concrete shape. The variable is written where only a privileged caller reaches and read in a revert on the path every holder takes, which is the whole of a family A exit gate.

Balance binding refuses to treat any `address => uint` mapping as a ledger. The bound mapping is the one that `balanceOf` reads, with a fallback for contracts that carry no ERC-20 interface at all, which was necessary for a set of Pied-Piper fixtures. Writes to it are classified as debit, credit, or set, with a separate determination of whether the written value is the raw transfer amount. This is what separates a hidden mint inside transfer, where the total credited exceeds the debited, from a normal transfer.

4.4 Rule catalogue

The catalogue holds 29 rule identifiers across the seven families, each with a base severity, and is a single source shared by the harness and the detector so that coverage can be computed against it. Every rule is a callable that takes the per-contract analysis context and returns findings carrying the contract, the function that holds the evidence, the source lines, a reasoning sentence written from the actual identifiers, and the discriminators that matched. Findings are a list; several rules may fire on one file, and deduplication is only on the tuple of rule, function, lines, and severity.

The table below gives one rule per family as an illustration; the complete table with every trigger is in the detector specification.

Rule	Family	Structural trigger	Base
EXIT_ADDR_GATE	A	An address-keyed boolean gate read in an end node on the transfer path, on a privileged-writable mapping	HIGH
BAL_PRIV_MINT	B	A privileged credit to the bound ledger or a privileged increase of the supply variable, outside the constructor	HIGH
LEAK_ARBITRARY_TRANSFERFROM	C	A privileged debit at a key that is not sender-dependent, with no allowance read dominating it	HIGH
OWN_HIDDEN_ROLE	D	An authorisation variable that gates a privileged function and is not exposed by any view	HIGH
STRUCT_DELEGATECALL_SETTABLE	E	A <code>delegatecall</code> whose target is privileged-writable, or is chosen from a parameter of a privileged function	HIGH

Rule	Family	Structural trigger	Base
HONEYPOT_LEGACY	F	Any of ten structural checks over the HoneyBadger techniques, each on an ETH-exit function	MED
PONZI_SHAPE	G	An ETH send to an address loaded from a store populated by a payable function, with no external value source	MED

Slither’s own high-impact detectors are run as an overlay and recorded as informational findings. They never drive a malicious verdict on their own. Only the seven checks whose shape is an exploit, among them `reentrancy-eth`, `arbitrary-send-eth`, `suicidal`, and `controlled-delegatecall`, are allowed to count at all.

4.5 Discriminators and the verdict ladder

The layer between findings and verdict is where the lookalikes of Section 3.3 are handled, and it distinguishes two classes that earlier iterations of this system conflated.

Bounding discriminators establish that the privilege is limited by code, or that it does not reach a user asset. A constant cap on a mint, a fee cap, a constant floor under an amount limit, a window bounded by an immutable expression, the existence of a symmetric un-gate, the absence of custody, a rescue restricted to foreign tokens, a two-step ownership handoff, a one-shot initializer, and a re-denomination write all fall here. A bounding discriminator demotes its finding to informational, so the finding is still reported as evidence and no longer contributes to the verdict.

Governance discriminators establish only who holds the power, not how much of it there is. A managed multi-role structure and an issuer-token shape fall here. They shape the reason text and the optional risk-type field, and they never change the verdict. Provenance, meaning whether a modifier came from a vendored library, is not a signal at all; treating it as one was an observed false negative, because a rug pull that inherits `Ownable` is still a rug pull.

The verdict is then a three-step ladder. A counting finding is one that is HIGH by base severity, or is one of five decisive medium rules, or is an exploit-shape overlay check, and that carries no bounding discriminator. If any counting finding exists, the verdict is malicious. Otherwise, if the transfer path depends on a call into a contract whose address the owner can set, the verdict is uncertain with the reason `external_dependency`, which is the one analytic abstention the policy retains. Otherwise the verdict is benign, with any informational findings reported as evidence of disclosed but bounded centralisation.

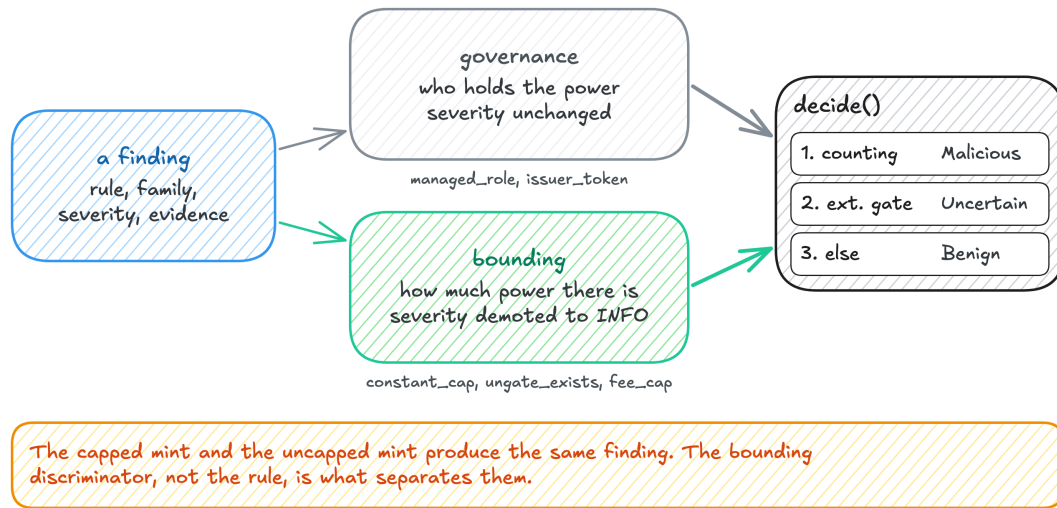


Figure 3: Bounding discriminators demote a finding to evidence and steer the file toward benign. Governance discriminators only annotate the reason.

The ladder is deliberate about abstention because the deployment grader treats abstention as a refusal to answer: a correct verdict scores plus one, a wrong verdict minus one, and an uncertain verdict zero [20]. A verdict class for “we noticed a medium-severity shape” is worth nothing under that rule, and an earlier version of this policy parked 106 honeypot files there. Section 6.5 measures what removing it did.

4.6 Robustness, determinism, and packaging

Three engineering properties are graded conditions rather than niceties, and each cost a measured defect. A bad file must never abort the batch. Compile failure, per-file timeout, and any analysis exception each resolve to an uncertain verdict with a reason and the batch continues. Compilation itself climbs a retry ladder: a constraint-aware `pragma` solver picks among the installed compiler versions, and on failure the ladder relaxes an exact `pragma`, tries the 0.4 through 0.7 series for files with no `pragma` at all, strips duplicate SPDX identifiers, and removes a byte-order mark. Of the 250 Tier 2 sources that did not compile when the corpus was ingested, the ladder rescued 63. The remainder fail for reasons no compiler version repairs: injected code that never compiled, dependency files absent from the dataset, and partial flattening. The reported run carries 189 compile failures on Tier 2 and one more on a Tier 1 fixture that is deliberately broken.

The ladder must not write to its input. The grader mounts the input directory read-only, and the first Docker run of the full corpus produced 48 files with an analysis error because the rewritten copy could not be created next to the source. The fix mirrors the input root into a scratch directory under the system temporary path, using hard links where the filesystem allows, so that relative imports and project remappings resolve exactly as they do in the original tree. After the fix the Docker run showed zero analysis errors and per-file verdicts identical to the local run.

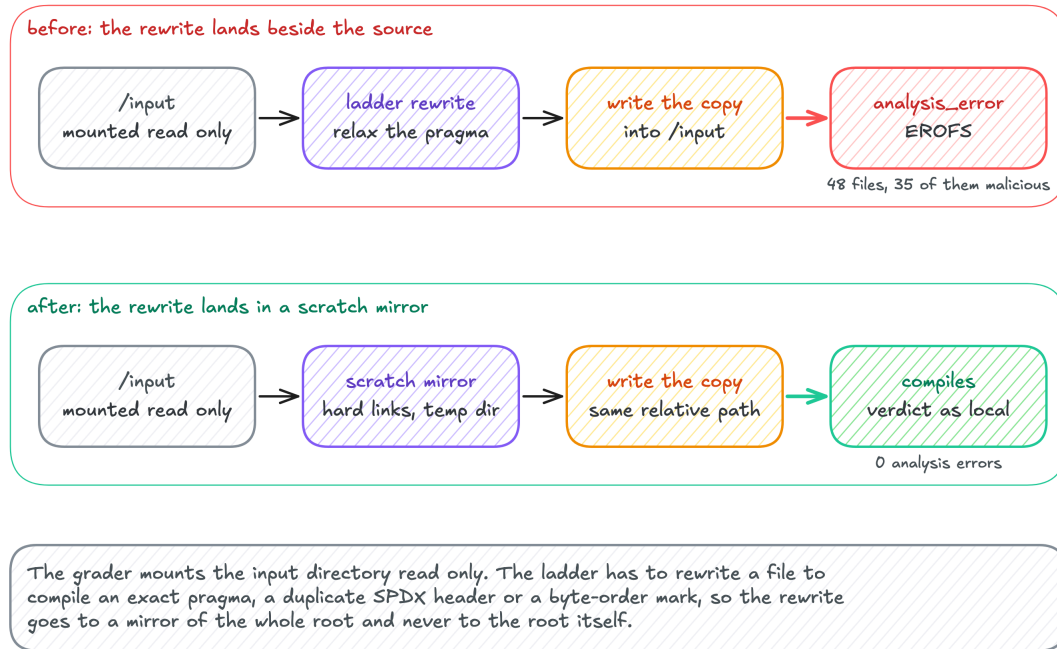


Figure 4: The retry ladder has to rewrite a source to compile it, and the grader mounts the input read-only. Mirroring the root and rewriting inside the mirror is what keeps a rescued file scoring the same under the grader as it does locally.

Output must be a pure function of input. The file walk, the contract and function iteration, and the finding order are all sorted, no timestamp is written, and representative picks inside rules are ordered by source line and then by name. A regression test runs the corpus under several hash seeds; before the fix it failed on three of them, because six files drifted in which of several equivalent findings they reported.

5. Benchmark Design

5.1 Harness contract

BAYBENCH treats a tool as a black box that reads a directory of `.sol` files and writes one `results.json`. Tools are registered by an entry that names either a Docker image or a shell command; tool code is never imported. Grading parity is fixed at `docker run --rm --network none`, so a network-dependent tool scores zero by construction, and a probe tool that attempts egress is kept in the registry to prove the isolation holds. Output is validated against a JSON Schema before scoring, and a schema violation fails the run with a message rather than producing a partial report.

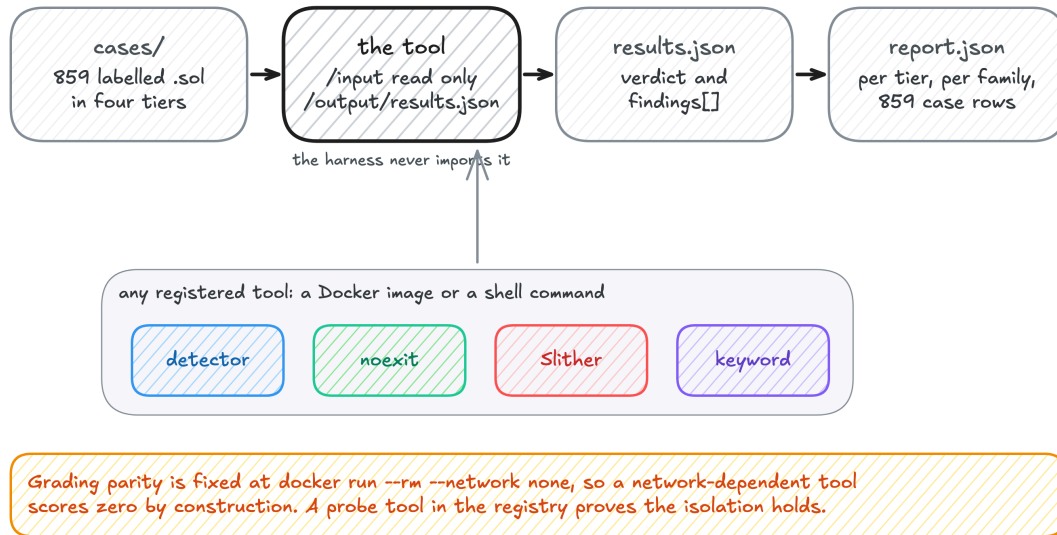


Figure 5: The tool is a black box behind one file contract, which is what lets four different implementations be scored identically and offline.

5.2 Corpus

The corpus holds 859 labelled Solidity files in four tiers. The tiers differ in provenance and in what a failure on them means, which is why they are scored separately.

Tier	n	Provenance	A failure here is
0	5	The organizers' public sample set, ingested with the labels stated in the file comments	disagreement with the grader's own boundary rules
1	67	Hand-written malicious and benign twins, one pair per catalogue rule, plus harness fixtures	a rule regression
2	779	Published paper fixtures from Pied-Piper, HoneyBadger, and CRPWarner	a missed pattern
3	8	Real and OpenZeppelin-derived contracts that look risky	an over-call

Tier 2 is the bulk and comes from three sources. Pied-Piper contributes 357 files, of which 200 are the backdoor-injected set and 157 are on-chain contracts, tagged by the paper's categories of arbitrary transfer, disable or freeze, and generate or destroy. HoneyBadger contributes 351 files tagged by technique, the largest groups being hidden state update with 164, inheritance disorder with 48, straw man contract with 34, uninitialised struct with 32, and balance disorder with 20. CRPWarner contributes 71 files, of which 19 are tagged hidden mint, 19 limiting sell, 3 leaking token, and 30 carry no category note in the manifest. Label distribution matters for reading the metrics. Tier 0 holds 2 benign and 3 malicious files. Tier 1 holds 35 benign, 30 malicious, and 2 uncertain. Tier 2 holds 529 malicious and 250 labelled uncertain, the latter being fixtures that did not compile at ingestion time. Tier 3 holds 5 benign and 3 malicious. Across the corpus, 551 cases carry both a malicious label and an expected family, and 548 of those carry exactly one family tag.

5.3 Label schema

Each case directory carries a `labels.yaml` with a preferred verdict, a non-empty list of accepted verdicts that must contain it, and optional expectations: the families whose rules should fire, the rule identifiers that would satisfy the case, and the function names that evidence should point at. The two-level verdict is what lets the corpus encode a defensible disagreement. Three Tier 3 files, the USDC fiat token, the Bancor smart token, and the Lido MiniMe token, carry a preferred verdict of malicious under the grader's boundary rules read literally, because each holds a blacklist, a burn-other, or an uncapped mint under a managed role with no code bound. The first two also accept a benign answer, since either decisive answer is defensible on a governance token and abstaining is not. The third accepts an uncertain answer instead, because its transfer path calls a controller address the role can change, which is the one condition under which this system is allowed to abstain.

5.4 Metrics

The primary metric is a verdict score per case: 1.0 for the preferred verdict, 0.75 for another accepted verdict, 0.5 for an uncertain verdict that was not accepted, and 0.0 otherwise. The partial credit for uncertainty is deliberate and is the one place where the benchmark is more forgiving than the deployment grader, which gives abstention nothing.

The headline number is not case-weighted. It is the mean of the per-tier means under weights of 1 for Tier 0, 2 for Tier 1, 1 for Tier 2, and 2 for Tier 3. Tier 2 dominates by count and would otherwise decide the score on its own, while five public samples would decide it if Tier 0 were weighted by importance rather than by size. A separate gate line reports Tier 0 exactness as a count out of five so that it stays visible.

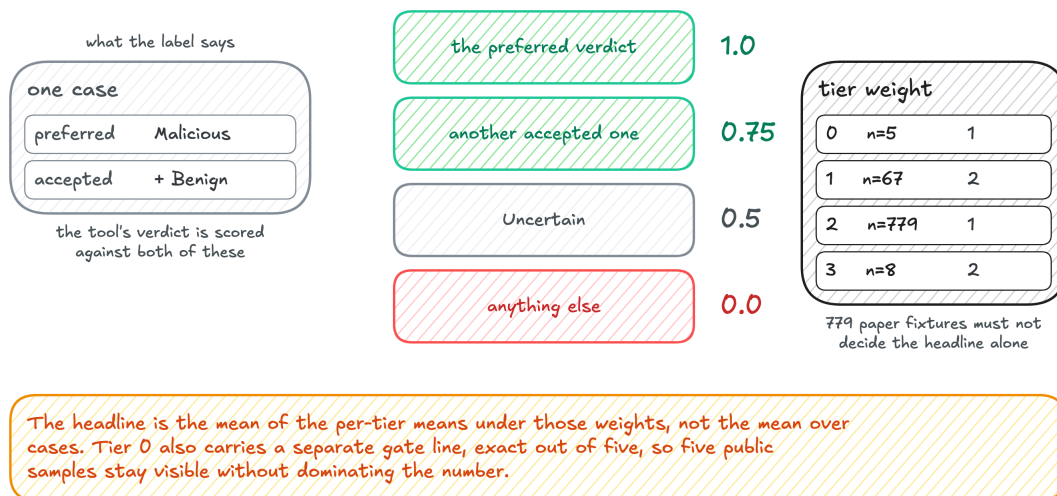


Figure 6: A case is scored by how close the verdict is to its label, and the tiers are weighted so that 779 paper fixtures do not decide the headline on their own.

Three secondary metrics are reported. Family recall is computed over malicious cases that carry expected families, and a case counts as a hit when at least one finding's family is in the expected set. It is any-hit, not all-hit, and Section 7.1 is about what that does and does not mean. HIGH false-positive rate is the share of benign-preferred cases carrying any HIGH finding, over 42 such cases. Evidence hit rate is the share of cases with expected functions where some finding points at one of them.

5.5 Baselines and the second implementation

Two baselines run on the same 859 files. The keyword baseline applies name regexes and calls a file malicious if any match, which approximates the industry checklist approach. The Slither baseline runs the analyzer natively and calls a file malicious if any high-impact detector fires, which is the strongest off-the-shelf answer available offline.

A second tool, `noexit`, is also reported. It is an independent implementation by the same team, written in TypeScript with 52 rule identifiers, registered only through the harness registry and run through the same grader. It is not an external baseline and should not be read as one. Its value here is as a second opinion on the same corpus from a different rule design: where the two tools agree on a file that the label calls malicious, the label is probably right, and where they both say benign, the label deserves a second look.

Aderyn and Mythril were not installed in the evaluation environment and do not appear in any table.

6. Evaluation

6.1 Setup

All numbers in this section come from one frozen run of each tool over all four tiers, executed locally on 2026-09-20 with the harness invoked as `bench run <tool> --no-docker --repeat 1`, against the corpus as it stood at commit `799361b`. The report committed at that commit carries, for each tool, both the summary metrics and one row per case, and `docs/research/_paper_stats.py` derives the confusion matrices and the attribution distribution below from those 859 rows. The corpus has since grown to 863 cases, four Tier 1 fixtures written for OpenZeppelin v5 shapes, and the detector’s weighted score on the larger corpus is unchanged at 0.9714. This paper reports the frozen 859-case run throughout rather than mixing the two, because the baselines have not been re-run on the larger corpus. The raw `results.json` files the runs produced are local work-directory artifacts and are not part of the repository. Wall-clock time for the full corpus was 304.8 seconds for the detector, 19.6 for `noexit`, 66.5 for Slither, and 0.38 for the keyword baseline; these are whole-run times, not per-contract latencies.

The detector’s weighted score in this local run is identical to the Docker run recorded at commit `799361b`, which executed under `--network none` with the input mounted read-only and with `--repeat 2`, took 381.9 seconds per repeat, and passed the determinism check.

6.2 RQ1: comparison on the same corpus

The four tools differ by a factor of more than two on the weighted score, and the ordering is the same on every component metric except uncertainty.

Tool	Weighted	Mean verdict	Family recall	HIGH-FP	Evidence hit	Compile fail	Tier 0 exact
detector	0.9714	0.9008	0.9201	0.0	1.0	190	5/5
noexit	0.9194	0.7989	0.6933	0.0238	0.9706	0	5/5
Slither (native high)	0.6298	0.6612	0.0036	0.119	0.0	0	3/5
keyword regex	0.4269	0.3513	0.1906	0.4286	0.0	0	3/5

Two entries in that table need reading carefully. Slither’s family recall of 0.0036 is not a claim that Slither found nothing; it is a mapping gap, because its high-impact detector names are not projected onto the A to G families, and a tool that reports a reentrancy cannot score a family-A hit by construction. The detector’s

190 compile failures are files it declined rather than files it got wrong, and they are counted at 0.5 through the uncertain rule; the two baselines report 0 because neither compiles anything.

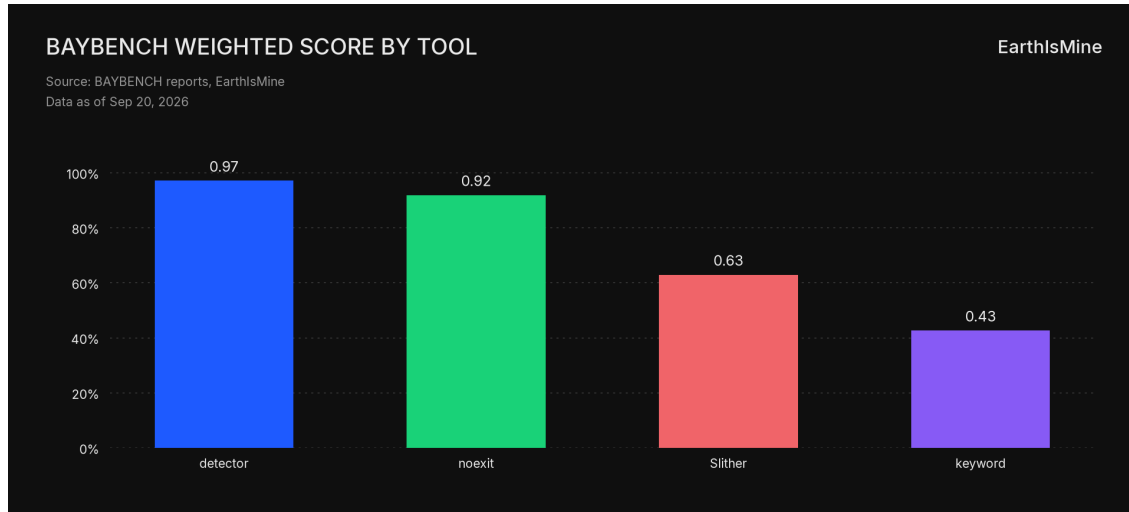


Figure 7: Tier-weighted score by tool on the 859-file corpus. Weights are 1 for Tier 0, 2 for Tier 1, 1 for Tier 2, 2 for Tier 3.

6.3 RQ2: where the accuracy comes from

Decomposing by tier separates three different claims: agreement with the grader’s own samples, rule coverage on minimal shapes, and generalisation to published corpora.

Tool	Tier 0 (n=5)	Tier 1 (n=67)	Tier 2 (n=779)	Tier 3 (n=8)
detector	1.0	1.0	0.8909	0.9688
noexit	1.0	0.8955	0.7875	0.9688
Slither	0.6	0.5672	0.6694	0.6875
keyword	0.6	0.597	0.3299	0.2188

The detector is exact on Tiers 0 and 1 and loses its points on Tier 2. The verdict confusion matrix shows where. Of the 529 Tier 2 files labelled malicious, 2 failed to compile and 527 were analysed; of those, 503 were called malicious, 18 benign, and 6 uncertain through the external-dependency abstention.

The 18 benign answers are the substantive misses and are triaged individually in the repository’s miss log. Eight of them are a labelling disagreement rather than a detection failure: each is a symmetric pause and unpauses or disables and enables pair, which the source corpus tags as a disable-or-freeze backdoor and the grader’s first boundary rule calls benign, and the second implementation returns benign on all eight as well. Several more are dataset artefacts, among them a frozen-account flag that the owner writes but that is never read on any transfer path, and a file whose injected backdoor is absent from the source. The remainder are genuine gaps and are listed as open in the log.

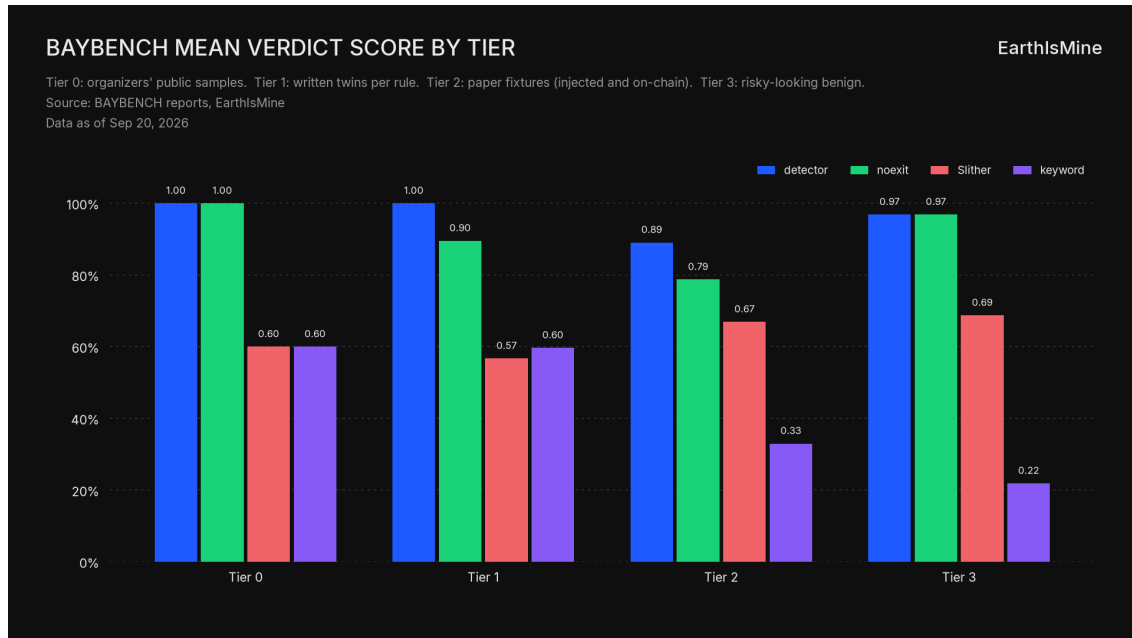


Figure 8: Mean verdict score by tier. The detector is exact on Tiers 0 and 1 and loses its points on the published-corpus tier. Per family, the detector’s recall over the 551 labelled-malice cases is 0.981 on exit gating ($n=105$), 0.8785 on balance tamper ($n=107$), 1.0 on leak ($n=8$), 1.0 on hidden owner ($n=4$), 1.0 on structure ($n=5$), 0.9105 on honeypot and drain ($n=324$), and 1.0 on Ponzi ($n=1$). The corresponding mean verdict scores are 0.9048, 0.965, 1.0, 1.0, 0.95, 0.9923, and 1.0. The families with four or five cases carry no statistical weight and are reported for completeness only.

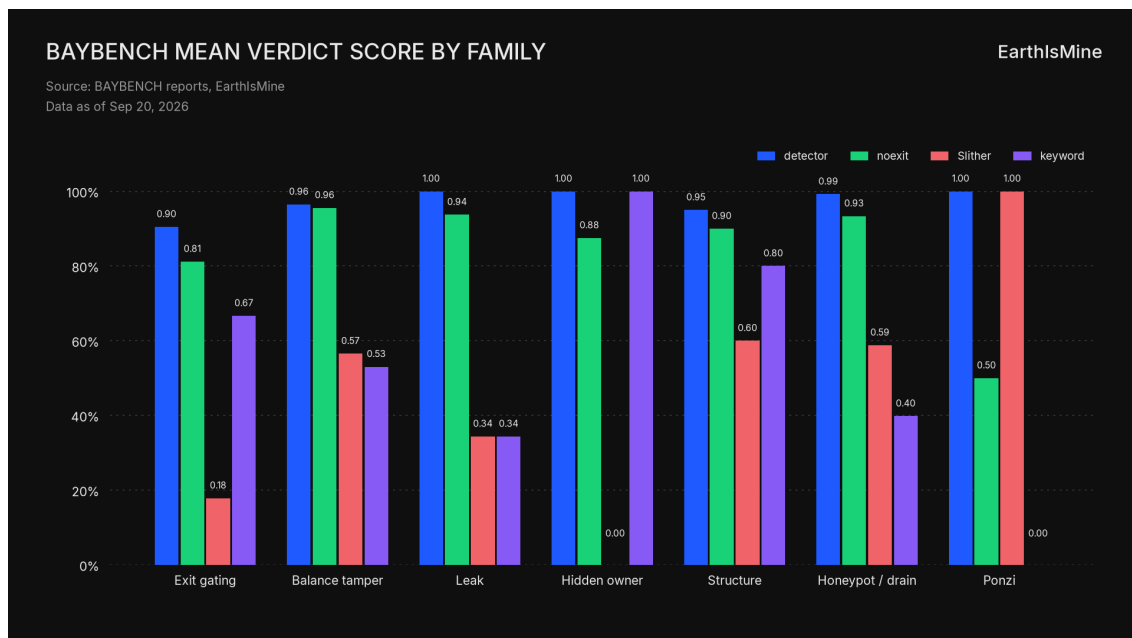


Figure 9: Mean verdict score by family. Families C, D, E, and G hold between one and eight cases each and are reported for completeness.

6.4 RQ3: behaviour on benign-looking contracts

The over-call brake is the HIGH false-positive rate, measured over the 42 benign-preferred files: 35 Tier 1 benign twins, the 2 benign public samples, and the 5 Tier 3 fixtures that must stay benign. The detector produces no HIGH finding on any of them, for a rate of 0.0. Native Slither is at 0.119 overall and 0.4 on Tier 3, and the keyword baseline is at 0.4286 overall and 1.0 on Tier 3, meaning it calls every risky-looking benign contract malicious.

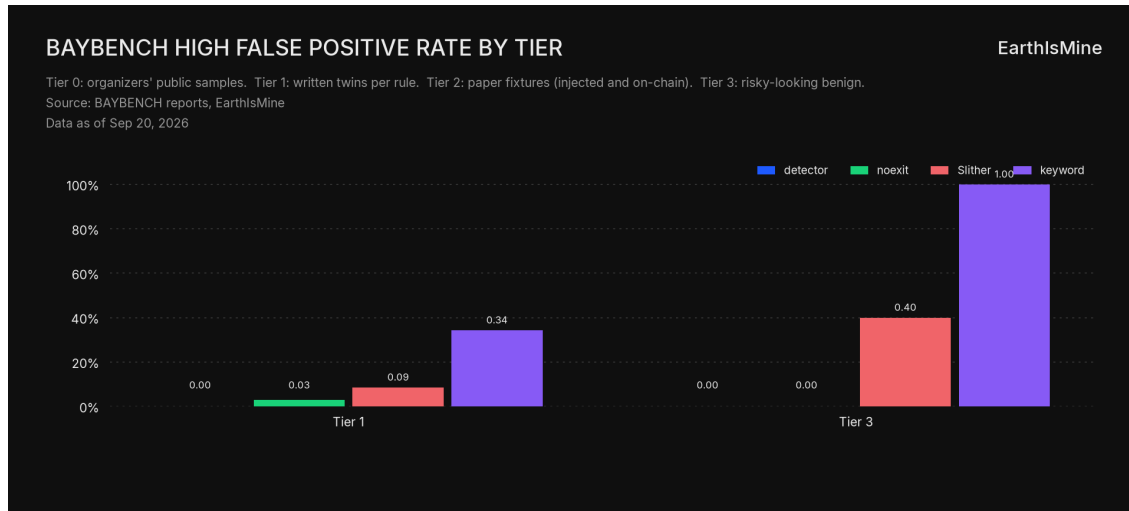


Figure 10: HIGH false-positive rate by tier. The keyword baseline flags every Tier 3 lookalike.

This is the metric that the bounding discriminators exist to protect, and it is the one that a deployment grader punishes hardest, since a wrong malicious verdict costs a point rather than merely failing to earn one.

6.5 RQ4: the decision policy, not the rule set

The largest single improvement in this system came from the verdict layer rather than from new detection. Under the previous policy, a single medium-severity finding resolved to uncertain. The honeypot rule was medium by base severity, so on 106 files the detector had already fired the correct rule and then abstained. That is a policy split, not a missing rule, and it is measurable because the repository’s miss log records each run.

Stage	Tier 2 weighted	Verdicts on the 527 compiling malicious-labelled files	Family F recall
First Tier 2 pass	0.7901	368 Malicious / 97 Uncertain / 62 Benign	0.031
After the analysis-layer pass	0.8209	373 Malicious / 141 Uncertain / 13 Benign	0.910
After the decisive policy	0.8909	503 Malicious / 6 Uncertain / 18 Benign	0.9105

The middle row is rule work: ten structural checks for the HoneyBadger techniques, a cycle guard, a shadowed-variable analysis, and a ledger-binding fallback, which took family F recall from 0.031 to 0.910. The bottom row is policy only. Removing the medium verdict class, splitting the discriminators into bounding and governance, and demoting the mutable-fee-recipient rule to informational moved 135 files out of uncertainty without changing a single trigger, 130 of them to malicious.

The public-sample count moved 3 of 5, then 4 of 5, then 5 of 5 across the same period, and the two closures had different causes. The `delegatecall` sample was closed by widening a trigger, from targets held in privileged-writable state to targets chosen from a parameter of a privileged function. The capped-mint

sample was closed by the policy alone: the finding is unchanged, and the constant-cap discriminator now demotes it instead of parking the file at uncertain.

The cost of the same change is visible in the table: benign answers on compiling malicious-labelled Tier 2 files rose from 13 to 18. Decisiveness is not free, and the design accepts five additional misses to convert 135 abstentions into answers.

6.6 RQ5: operational constraints

The detector is deterministic in the sense the benchmark checks. Two separate local invocations produced byte-identical output files, and the Docker double run at commit 799361b passed the harness determinism comparison with no diffs. Both local runs hash to the same digest:

```
0c2e7b73bd1c2e2450cd6121fc6fc455feff42d0a361fe4444b1d2d1ab2130de
```

The full corpus completes in 382 seconds per repeat inside the container with networking disabled. That figure was measured on an arm64 host running the image under emulation, and the deployment budget is ten minutes on two x86 virtual CPUs, so it indicates the order of magnitude rather than the headroom on the target machine. The tool ships a submission mode that prints a single schema-valid JSON array to standard output with all logging on standard error, and holds a wall-clock budget after which the remaining files are emitted as uncertain so that a complete array always reaches the grader. The repository's test suite collects 962 tests.

6.7 The public sample set

The organizers' five public samples were ingested as Tier 0 with the labels stated in their file comments. Both team tools are exact on all five.

File	Label	Basis for the verdict	Detector
P1 Standard token	BENIGN	No privileged writer reaches balances, gates, or a value sink	Benign
P2 Hidden mint	MALICIOUS	Privileged supply increase with no code-enforced bound	Malicious
P3 Honeygot	MALICIOUS	Owner-written allowlist read in a revert on the transfer path	Malicious
P4 Capped mint	BENIGN	Same mint shape as P2, demoted by the constant-cap discriminator	Benign
P5 Delegatecall backdoor	MALICIOUS	delegatecall to a target chosen from a parameter of a privileged function	Malicious

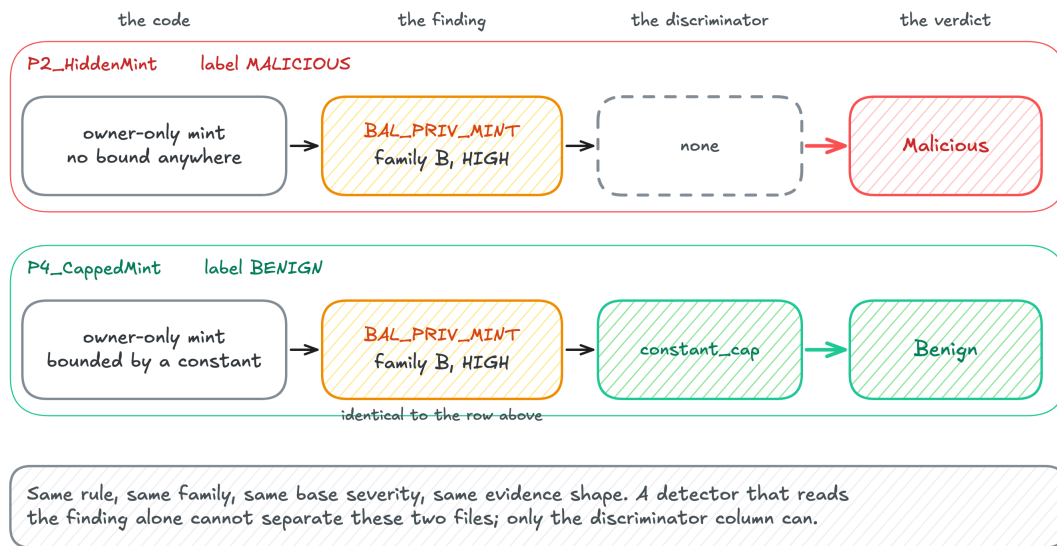


Figure 11: The rule output is identical on both files; only the bounding discriminator differs.

The P2 and P4 pair is the sharpest test in the set, because the two files produce the same finding and require different verdicts. The discriminator layer, not the rule, is what separates them. The file labelled P3 Honeypot is an owner allowlist on transfer, a family A exit gate rather than a HoneyBadger bait-and-trap, which is worth stating because the name suggests otherwise.

7. Discussion

7.1 Family recall is not multi-pattern attribution

The detector appends every matching rule, so a file that is a rug pull under three descriptions produces findings in three families. On the 551 labelled-malice cases it fires three or more families on 312 of them, that is 56.6 percent, with a mean of 2.82 families fired and 1.89 families beyond the one the source paper tagged. Of those 312 cases, 307 still resolve to malicious.

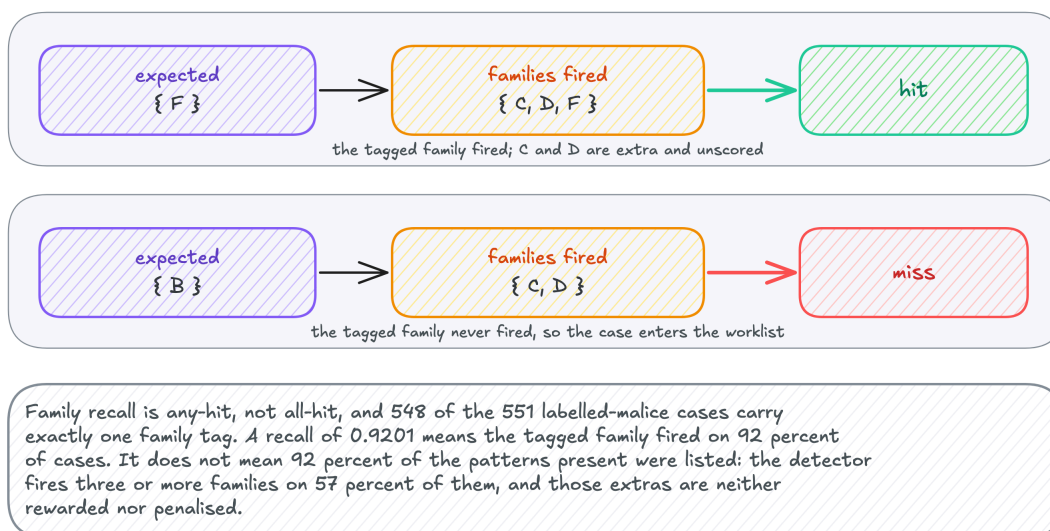


Figure 12: Family recall asks only whether one expected family fired. Extra families neither earn credit nor cost any, which is why the metric cannot speak to multi-pattern attribution.

Two readings of that are available and only one is supported. It is over-attribution of findings, which the benchmark does not penalise and the HIGH false-positive rate does not see, because these files are not benign. It is not evidence that the detector enumerates every pattern present in a contract, because 548 of the 551 labels carry exactly one family tag and the metric is any-hit. A family recall of 0.9201 means that on 92 percent of labelled-malice cases at least one finding landed in the family the source paper named. It does not mean that 92 percent of the patterns present were listed.

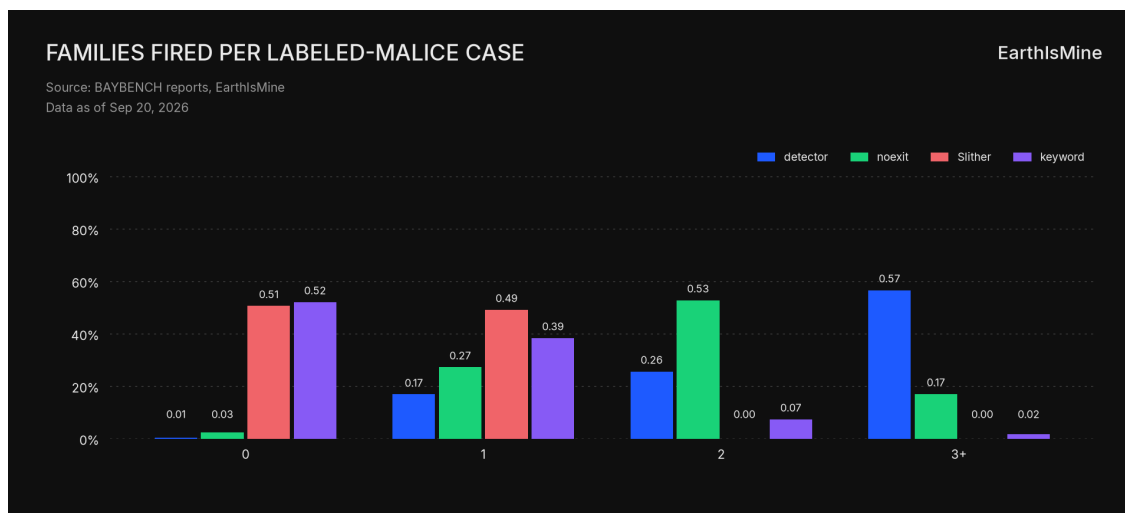


Figure 13: Distribution of the number of families fired per labelled-malice case.

The extra families are frequently the same trick under a finer split rather than a second trick. A Honey-Badger withdraw function guarded by an owner comparison is tagged F by the paper and also fires C and D here; a Pied-Piper destroy function is tagged B and also fires C and D. Genuine compounds exist in the corpus and are unlabelled. The iterate worklist queues only cases where no required family fired, so extra findings never enter it.

7.2 *Two abstention semantics*

The benchmark and the deployment grader use the same three words and do not price them the same way. In BAYBENCH an uncertain verdict earns 0.5, or 0.75 when the label accepts it, which is appropriate for a development instrument that should reward a tool for knowing what it cannot see. In the deployment grader an uncertain verdict earns nothing at all while a wrong verdict costs a point, so uncertainty is a refusal to play rather than a hedge [20]. A policy tuned on the first and deployed under the second will lose points it already earned, which is the failure mode the decisive ladder in Section 4.5 was designed against. The residual abstentions the detector emits are therefore restricted to four causes that are all genuine blindness: compile failure, timeout, analysis error, and a transfer path that calls into a contract the file does not contain.

7.3 *Why the baselines behave as they do*

The keyword baseline is not a straw man; it is the approach the industry checklists expose as an API and the approach the track brief explicitly rules out. Its profile shows why. It reaches 0.597 on Tier 1, where the fixtures are minimal and the names are honest, and collapses to 0.3299 on Tier 2 and 0.2188 on Tier 3, where names are adversarial in the first case and legitimate in the second. Its HIGH false-positive rate of 1.0 on Tier 3 is the whole argument in one number.

Native Slither behaves in the opposite way. It is a competent vulnerability analyzer being asked a question about intent. Its verdict score of 0.6694 on Tier 2 is mostly the credit structure rather than detection: a high-impact finding on a file that is in fact malicious is right for the wrong reason, and its family recall of 0.0036 shows it is not identifying the scam pattern. Its Tier 3 rate of 0.4 confirms that high-impact vulnerability findings are not a proxy for malice.

8. Threats to Validity

8.1 *Construct validity*

The Tier 1 fixtures were written by the same team that wrote the rules, one malicious and one benign twin per rule identifier. A Tier 1 rule recall of 1.0 is therefore a unit-test floor, not evidence of generalisation, and should be read as a regression guarantee rather than as a result. Any claim about generalisation rests on Tiers 2 and 3.

Several corpus labels were amended in the same change set as the decision policy, on 2026-09-20: the Tier 1 malicious twins of the five decisive medium rules, the mutable-fee-recipient twin, a re-shaped benign twin for the privileged-role rule, and the three Tier 3 governance tokens. Each amendment cites a stated boundary rule from the grader's brief, and each is recorded with its rationale in the miss log, but a reader is entitled to discount Tier 1 and Tier 3 accordingly. Tier 2 splits in two here. Its family tags come from the source papers and were not amended. Its verdict labels are the team's, assigned at ingestion, and the 250 files labelled as accepting only an uncertain verdict are an ingestion-time decision rather than a paper one, which is the artefact quantified below.

The benchmark does not compute classical precision, recall, or F1 over verdicts, and this paper does not use those words for its own numbers. The confusion matrices in Section 6.3 are provided so that a reader who wants them can compute them from the same run.

8.2 Internal validity

A known label artefact costs the detector roughly 0.08 of the Tier 2 mean. At ingestion, 250 Tier 2 fixtures did not compile under the pinned compiler and were labelled as accepting only an uncertain verdict. The compile ladder later rescued 63 of them, and the detector then returns the source paper’s own verdict, 60 malicious and 3 benign, each of which scores zero against a label that accepts only uncertainty. The artefact is tool-neutral, since the second implementation shows the same pattern, and a label amendment is proposed rather than applied, because the corpus labels are owner-ratified [proposal: amend the non-compiling Tier 2 rows to accept the paper verdict alongside uncertain].

One harness hazard was observed and recorded. A concurrent benchmark invocation re-staged the shared per-tool working directory under a running iteration, and the detector correctly abstained with an analysis error on 426 files whose inputs had vanished. The reported run was executed with no concurrent invocation; the fix, a per-invocation working directory or a lock, is an open harness item.

8.3 External validity

The five public samples are explicitly not the grading set; the held-out set is described by the organizers as variants of these five plus additional patterns, including benign contracts that look dangerous and malicious contracts that look ordinary. A Tier 0 result of 5 of 5 is a calibration check, not a prediction.

Tier 2 has a strong composition bias. Of 779 files, 351 come from a honeypot corpus whose techniques are concentrated in Solidity 0.4-era code, and 200 are synthetically injected backdoors. Accuracy on that mixture is not evidence of accuracy on contemporary DeFi contracts, and family F, with 324 labelled-malice cases, dominates any aggregate that is not decomposed.

The accuracy figures of the systems surveyed in Section 2 are measured on their own corpora with their own labels and are not comparable with the numbers here. No table in this paper places them side by side. Finally, this evaluation scores the benchmark output format. The deployment surface is a single JSON array on standard output with uppercase verdicts, basename file fields, and mandatory evidence on every malicious verdict. That adapter exists and is schema-validated on the public set, but a full-corpus pass through the deployment format is not what Section 6 measures.

9. Conclusion

The structural question that Section 4.1 states is the whole design, and it survived contact with 859 files. A privileged writer, a state variable that writer controls, and an impact on the user’s exit path or ledger are sufficient to express 29 rule identifiers across seven families of published scam patterns, and are expressible on a source-level intermediate representation without ever comparing a user-chosen identifier against a word list.

What this evaluation supports: on this corpus, with these labels, the structural detector reaches a tier-weighted score of 0.9714 against 0.6298 for native high-impact static analysis and 0.4269 for name matching; it produces no HIGH finding on any of the 42 benign-preferred files; it is exact on the grader’s own public samples; and it is deterministic and offline under the grading container. It also supports a narrower claim that is more useful for future work: the last and largest improvement came from the verdict layer rather than from detection, and separating bounded privilege from governed privilege is what made a decisive answer safe to give.

What it does not support: that a family recall of 0.9201 enumerates the patterns in a contract, that a hand-written twin corpus measures generalisation, or that five public samples predict a held-out set. The corpus composition, the label amendments, and the abstention artefact in Section 8 are the specific places where the headline number is softer than it looks.

References

Titles follow the wording of this repository’s literature survey, which is the source each entry was taken from. Full published titles are not expanded here because they were not verified against the sources offline [check: expand titles from the linked sources before external release].

[1] The Art of the Scam (HoneyBadger) · Torres, Steichen, State · USENIX Security 2019

[PDF](#), [arXiv](#)

First honeypot taxonomy, eight techniques in three levels. Symbolic execution over bytecode; 690 honeypots found, 87 percent precision on manual validation. Appendix Table 5 lists 24 named honeypots with addresses.

[2] Pied-Piper · Ma et al. · ACM TOSEM 2022

[ACM](#), [repository](#)

Five ERC-token backdoors, Datalog with directed fuzzing, 189 confirmed in 13,484 contracts, four CVEs, and a released set of 200 backdoor-injected contracts.

[3] TokenScope · Chen et al. · ACM CCS 2019

[PDF](#)

Detects inconsistency between storage changes, interface behaviour, and emitted events; precision above 99.9 percent.

[4] CRPWarner · Lin et al. · IEEE TSE 2024

[arXiv](#), [repository](#) and [datasets](#)

103 real rug pulls reduced to hidden mint, limiting sell order, and leaking token, expressed as Datalog rules over bytecode. Precision 91.8, recall 85.9, F1 88.7 on 69 open-source scam contracts.

[5] Stop Pulling my Rug (Tokeer) · Zhou, Sun, Ma, Chen, Yan, Jiang · ICSE-SEIP 2024

[PDF](#)

201 incidents totalling 425 million dollars; four risks with base rates, blacklist 58.7 percent and balance modification 42.3 percent. Models transfer as four sub-processes with five matching plugins; 98.0 recall, 98.9 precision.

[6] From Programming Bugs to Multimillion-Dollar Scams (TrapdoorAnalyser) · Huynh et al. · 2023, revised December 2024

[arXiv](#)

1,859 manually inspected trapdoor tokens, five techniques, semantic checking built on the Slither Python API; dataset of roughly 30 thousand labelled UniswapV2 tokens.

[7] Your Token Becomes Worthless (RPHunter) · Wu et al. · 2025

[arXiv](#), [dataset](#)

645 rug pulls with source; code-risk taxonomy of three categories and eight sub-types with counts.

[8] Detecting Rug-Pull via balance tracking · Applied Sciences 2025

[MDPI](#)

Tracks per-function balance changes on bytecode across six backdoor types; 0.98 precision, 0.96 recall.

[9] Geth-based ERC20 honeypot detector · Discover Computing 2025

[Springer](#)

Taint analysis over boolean storage indexed by sender or recipient that decides a revert branch; misses the backdoors that never touch the transfer function.

[10] SoK: A Taxonomic Analysis of DeFi Rug Pulls · Sun, Ma, Nie, Liu · ISSTA 2025

[PDF](#)

31 papers and 27 industry sources merged into 35 rug-pull types; 13 tools evaluated at 25.7 to 62.9 percent coverage, 9 types detected by none, best tool at 31.3 percent on compound scams. Names four failure modes of static tools.

[11] SoK: Rug Pull Causes, Datasets, Tools · 2024

[arXiv](#)

34 root causes and the list of types no evaluated tool covered.

[12] ObfProbe: Obfuscated Funds Transfers · 2025

[arXiv](#)

Seven bytecode-level obfuscation features over 1.03 million contracts; a state-of-the-art Ponzi detector falls from 79 percent to 12 percent under obfuscation.

[13] Static analysis of NFT rug pulls · 2025

[arXiv](#)

Slither applied at scale over 49.9 thousand NFT contracts; reports which constructs dominate flagged contracts.

[14] Trade or Trick? · Xia et al. · ACM SIGMETRICS 2022

[arXiv](#)

10,920 scam tokens, roughly half of Uniswap V2 listings in the studied window.

[15] Token Spammers, Rug Pulls, and Sniper Bots · Cernera et al. · USENIX Security 2023

[page](#)

60 percent of tokens live less than one day; one-day rug pulls estimated at 240 million dollars.

[16] Do Not Rug On Me · Mazorra, Adan, Daza · 2022

[arXiv](#)

Machine learning over pool, holder, and transaction-graph features across 27,588 tokens.

[17] Detecting Ponzi Schemes on Ethereum · Chen et al. · WWW 2018

[PDF](#)

Opcode-frequency and account features with gradient boosting.

[18] GoPlus Token Security API · industry reference

[documentation](#)

Boolean token-security fields including honeypot, hidden owner, mintable, modifiable slippage, transfer pausable, blacklist, proxy, external call, and owner-changeable balance.

[19] TokenSniffer Exploit Typologies · industry reference

[documentation](#)

Exploit typologies and the corresponding tests for proxy, pausable, mint, ownership restoration, maximum transaction amount, modifiable fee, and blacklist.

[20] TRUST404 Track 1: Smart Contract Threat Detection, public sample set and grading rules · organizers · 2026

[track page](#)

Task brief, output schema, boundary rules for the benign and malicious classes, and the discrimination score of plus one for a correct verdict, zero for uncertain, and minus one for a wrong verdict. Vendored in this repository at `public set/challenge_public/` and `docs/judge/challenge_public/`.

[21] Slither · static analysis framework, `slither-analyzer` 0.11.6

[repository](#)

The parser and intermediate representation this detector is built on, and, run natively, the high-impact baseline in Section 6.